LAB ASSIGNMENT & SUBMISSION WORKSHEET

Course Code & Name: Database Management Systems (DBMS) Lab

Lab Session: Lab 1 – Installation, Basic Setup & DML Operations (INSERT, UPDATE, DELETE)

Student Name: Raj Jadav

SRN: 2613600165 **Date:** 09/11/2026

Instructions for Students

1. Open the MySQL Command Line Client on your system.
2. Create a database named `student_lab_db` and switch to it before starting.
3. Execute the SQL queries sequentially as asked in the tasks below.
4. Paste the precise SQL code used and the expected output table/result in the designated spaces.

Task 1: Table Creation & Setup (DDL)

Create a table named **Products** in your database with the specified schema: ProductID (Integer, Primary Key, Auto Increment), ProductName (Varchar 50, Not Null), Category (Varchar 30), Price (Decimal 10,2), StockQuantity (Integer).

Q1. Write the SQL statement to create the Products table and describe its structure.

```
CREATE DATABASE IF NOT EXISTS student_lab_db;
USE student_lab_db;

CREATE TABLE Products (
    ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ProductName VARCHAR(50) NOT NULL,
    Category VARCHAR(30),
    Price DECIMAL(10,2),
    StockQuantity INT
);

DESCRIBE Products;
```

Expected output — DESCRIBE Products;

Field	Type	Null	Key	Default	Extra
ProductID	int	NO	PRI	NULL	auto_increment
ProductName	varchar(50)	NO		NULL	
Category	varchar(30)	YES		NULL	
Price	decimal(10,2)	YES		NULL	
StockQuantity	int	YES		NULL	

Task 2: Data Insertion (INSERT)

Perform the requested data population steps into the Products table.

Q2. Write the SQL queries executed for insertion and verification.

```
-- Insert a single product
INSERT INTO Products
    (ProductName, Category, Price, StockQuantity)
VALUES
    ('Laptop', 'Electronics', 55000.00, 15);

-- Insert three more products in one statement
INSERT INTO Products
    (ProductName, Category, Price, StockQuantity)
VALUES
    ('Wireless Mouse', 'Electronics', 800.00, 50),
    ('Office Chair', 'Furniture', 4500.00, 20),
    ('USB Cable', 'Electronics', 250.00, 100);

SELECT * FROM Products;
```

Expected output — initial records

ProductID	ProductName	Category	Price	StockQuantity
1	Laptop	Electronics	55000.00	15
2	Wireless Mouse	Electronics	800.00	50
3	Office Chair	Furniture	4500.00	20
4	USB Cable	Electronics	250.00	100

The ProductID values assume a fresh table and inserts executed in the order shown.

Task 3: Data Modification (UPDATE)

Q3. Write the SQL queries executed for the updates.

```
-- 1. Increase Wireless Mouse price to 950.00
UPDATE Products
SET Price = 950.00
WHERE ProductID = 2;

-- 2. Add 10 units to all Electronics products
UPDATE Products
SET StockQuantity = StockQuantity + 10
WHERE Category = 'Electronics';

-- Verify updated records
SELECT * FROM Products;
```

Expected output — after updates

ProductID	ProductName	Category	Price	StockQuantity
1	Laptop	Electronics	55000.00	25
2	Wireless Mouse	Electronics	950.00	60
3	Office Chair	Furniture	4500.00	20
4	USB Cable	Electronics	250.00	110

If your ProductID values differ, run SELECT * FROM Products and use the actual ID for Wireless Mouse.

Task 4: Data Deletion (DELETE)

Perform the requested record cleanup tasks.

Q4. Write the SQL queries executed for deletion.

```
-- 1. Delete Office Chair
DELETE FROM Products
WHERE ProductName = 'Office Chair';

-- 2. Delete products priced below 300.00
DELETE FROM Products
WHERE Price < 300.00;

-- 3. Display final remaining records
SELECT * FROM Products;
```

Expected output — final remaining records

ProductID	ProductName	Category	Price	StockQuantity
1	Laptop	Electronics	55000.00	25
2	Wireless Mouse	Electronics	950.00	60

Office Chair is deleted by name. USB Cable is deleted because its price (250.00) is less than 300.00.

Task 5: Conceptual & Error Analysis

1. What happens if you execute an UPDATE statement without including a WHERE clause?

Answer: All rows in the table are updated because no condition restricts which records are affected. A WHERE clause should be used to modify only selected rows.

2. What error code or message do you receive if you attempt to insert a duplicate value into a PRIMARY KEY column?

Answer: MySQL returns error code **1062** (SQLSTATE 23000), indicating a duplicate entry for a primary key. The exact key name in the message may vary by MySQL version.

```
-- Example: fails if ProductID 1 already exists
INSERT INTO Products
    (ProductID, ProductName, Category, Price, StockQuantity)
VALUES
    (1, 'Keyboard', 'Electronics', 1200.00, 30);

-- Typical message:
ERROR 1062 (23000): Duplicate entry '1' for key 'products.PRIMARY'
```

Submission note: Output tables show expected results. Execute the queries in MySQL and attach actual screenshots if your instructor requires execution evidence.